PES UNIVERSITY

Department of Computer Science and Engineering

SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT

Software Architecture and Design Specification

Electronics Store Inventory Management System (ESIMS)

Project	Electronics Store Inventory Management System (ESIMS)
Group / Project ID	Team 8
Version	1.0
Date	30-09-2026
Technology	Python (Django), PostgreSQL
Methodology	Agile

Revision History

Version	Date	Author	Change Summary
1.0	30-09-2026	Team 8	Architecture, UML, API, security and design completed from approved SRS.

Approvals

Role	Name	Signature / Date
Team / Faculty Reviewer	Team 8	

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification defines the architecture, major components, interfaces, data design, security architecture and detailed interaction design for the Electronics Store Inventory Management System (ESIMS), based on the approved SRS.

1.2 Scope

The design covers authentication and role management, product/category catalog management, stock-in and stock-out transactions, supplier management, low-stock alerts, inventory/sales reporting and inventory search/filtering. It does not introduce the out-of-scope integrations listed in the SRS.

1.3 Audience

Developers, testers, faculty/evaluators, maintainers and security reviewers.

1.4 Definitions

ESIMS – Electronics Store Inventory Management System; RBAC – Role-Based Access Control; SKU – Stock Keeping Unit; Stock-in – purchase transaction increasing inventory; Stock-out – sales transaction decreasing inventory; NFR – Non-Functional Requirement; RTM – Requirements Traceability Matrix.

2. Document Overview

2.1 How to Use this Document

Sections 3 and 4 describe the architecture and detailed design respectively. The diagrams show component responsibilities, domain relationships, runtime deployment and key request flows. Traceability connects the design back to SRS requirements.

2.2 Related Documents

- ESIMS Software Requirements Specification, Version 1.0 (approved).
- ESIMS Software Test Plan, Version 1.0.
- PES University Software Engineering Level 3 Student Submission Guide / Coffee Kiosk exemplar.

3. Architecture

3.1 Goals & Constraints

- Maintain accurate and auditable stock quantities and prevent negative stock.
- Enforce Staff/Administrator role separation and protect credentials and sensitive data.
- Support the SRS performance target: 95% of catalog searches within 2 seconds under normal load.
- Support at least 99% availability during defined business hours, excluding planned maintenance.
- Keep stock-in/stock-out workflows within four screens/steps for Staff.
- Use Python/Django with PostgreSQL, Git/GitHub, automated tests and CI as specified by the SRS.
- Single-store deployment and no public self-registration are constraints for Version 1.0.

3.2 Stakeholders & Concerns

Stakeholder	Primary architectural concern
Staff	Fast, clear catalog and stock transaction workflows; access only to permitted functions.
Administrator	Complete management of products, categories, suppliers, staff accounts and reports.
Developers	Modularity, testability, maintainability and clear separation of concerns.
Faculty / Evaluators	Requirement coverage, UML/design completeness, traceability and security validation.

3.3 Component (UML) Diagram

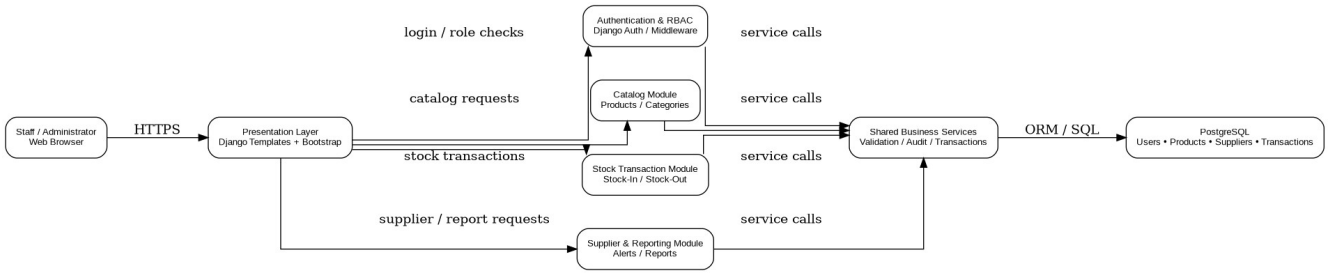


Figure 1. ESIMS component architecture. The browser communicates with the Django presentation layer over HTTPS; feature modules use shared business services and PostgreSQL persistence.

3.4 Component Descriptions

Component	Responsibilities	Key interfaces	Requirements
Presentation Layer	Server-rendered pages/forms, navigation, validation messages and role-driven views.	HTTPS requests; Django views/templates	FR-001–FR-019, NFR-003
Authentication & RBAC	Login, password verification, session lifecycle and role checks.	Login service; authorization middleware	FR-001–FR-004, SEC-001/002/004
Catalog Module	Product/category CRUD, search/filter, active/disabled status and stock display.	Product/category services	FR-005–FR-009
Stock Transaction Module	Atomic stock-in/out, quantity validation, transaction IDs and audit records.	Stock service; transaction repository	FR-010–FR-014, NFR-004
Supplier & Reporting Module	Supplier CRUD, low-stock alerts and operational reports.	Supplier/report/alert services	FR-015–FR-019
PostgreSQL Data Layer	Persistent users, products, categories, suppliers, transactions and audit/alert data.	Django ORM / SQL	NFR-005/006, SEC-005/006

3.5 Chosen Architecture Pattern and Rationale

A layered architecture with modular Django applications is selected. The presentation layer handles HTTP/UI concerns; authentication/RBAC handles identity and authorization; feature modules implement business rules; shared services enforce reusable transaction and validation logic; the data layer isolates PostgreSQL persistence. This matches the SRS description of a self-contained Django application and supports the maintainability/testability goals.

A microservices architecture is not selected for Version 1.0 because the SRS describes a single-location, self-contained application with no external e-commerce/payment integrations. Additional service decomposition can be considered if deployment scale or integration scope changes.

3.6 Technology Stack & Data Stores

Layer	Technology	Purpose
Web application	Python, Django 5.x	Routing, views, ORM, authentication and business application.

UI	Django Templates + Bootstrap 5	Server-rendered, role-driven web interface.
Database	PostgreSQL 15.x	Relational persistence and transaction integrity.
Optional API	Django REST Framework 3.x	API layer if a decoupled frontend is used.
Testing / CI	Django test framework or pytest; GitHub Actions	Automated tests and continuous integration.
Design / API tools	Draw.io / PlantUML; Postman	Architecture/UML diagrams and API validation.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Concurrent stock updates	Incorrect inventory quantity or duplicate transactions.	Use database transactions/atomic updates, validation and unique transaction IDs; test double submission.
Unauthorized admin access	Exposure or modification of sensitive data.	Central RBAC middleware, server-side authorization tests and least-privilege checks.
Database/server outage	Unavailable inventory functions.	Operational monitoring, database backup procedure and clear recovery steps.
Slow catalog search	Failure of NFR-001.	Indexed search fields, query review and JMeter/load testing.
Invalid input	Data corruption or application errors.	Server-side validation, sanitization and negative tests.

3.8 Traceability to Requirements

SRS requirement	Architecture ref	Design element	Test reference
FR-001 Login	AUTH-01	SEQ-01; Auth component	UT-AUTH-01 / ST-AUTH-01
FR-005 Add product	CATALOG-01	Product service; Class Product	UT-CAT-01 / ST-CAT-01
FR-010 Stock-in	STOCK-01	Stock service; transaction model	IT-STOCK-01 / ST-STOCK-01
FR-012 Reject over-sale	STOCK-02	Stock validation rule	UT-STOCK-02 / ST-STOCK-02
FR-017 Low-stock alert	REPORT-01	Alert service; SEQ-03	UT-ALERT-01 / ST-ALERT-01
SEC-002 RBAC	AUTH-02	Authorization middleware	UT-SEC-01 / ST-SEC-01
NFR-001 Search response	API-01	Search API/service design	PERF-01 / ST-PERF-01

3.9 Security Architecture

Security objectives are to protect credentials and sensitive internal data and to preserve the integrity/accountability of stock transactions. The architecture applies defense in depth across authentication, authorization, transport, validation, persistence and audit logging.

STRIDE concern	ESIMS control	Relevant SRS requirements
Spoofing	Django authentication, password hashing, session expiry	SEC-001, SEC-004
Tampering	Server-side validation, CSRF protection, atomic stock updates, database constraints	SEC-003, FR-012, NFR-004
Repudiation	Audit trail containing user, timestamp and quantity; security-event logging	FR-014, SEC-006
Information disclosure	HTTPS, authorization checks, protected supplier/pricing data and non-sensitive logs	SEC-005
Denial of service	Input limits/validation; rate limiting for externally exposed APIs; load testing	NFR-001
Elevation of privilege	Central RBAC middleware and admin-only views/actions	FR-002, SEC-002

4. Design

4.1 Design Overview

The design decomposes ESIMS into independently testable Django modules while keeping shared business rules in service functions. Product, Category, Supplier, User and StockTransaction are the principal domain entities. Stock-

changing operations are treated as atomic transactions so that inventory quantity and transaction history remain consistent.

4.1.1 Module Decomposition

Module	Responsibilities	Dependencies	Owner / functional area
Authentication	Login, roles, session expiry, staff-account management	Django Auth, User	Roopa Sreedhar A
Catalog	Products, categories, search/filter, active status	Product/Category models	Sakthevel Saravanan
Stock	Stock-in/out, validation, IDs, audit trail	Product, Supplier, User, Transaction	Samiksha B N
Supplier & Reporting	Supplier CRUD, alerts, stock/sales reports	Supplier, Product, Transaction	Rishitth G R

4.1.2 Domain / Class Diagram

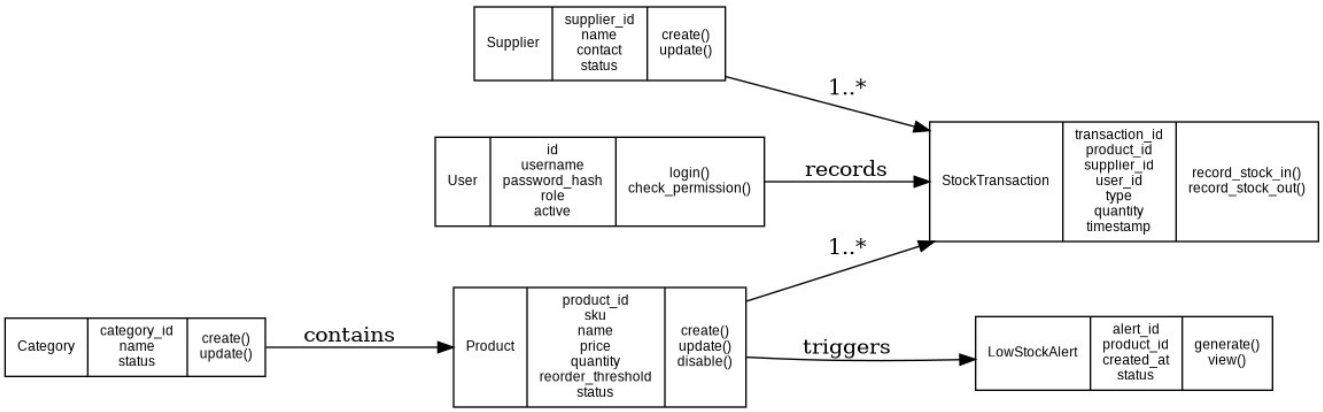


Figure 2. Major ESIMS domain classes and relationships.

4.2 UML Sequence Diagrams

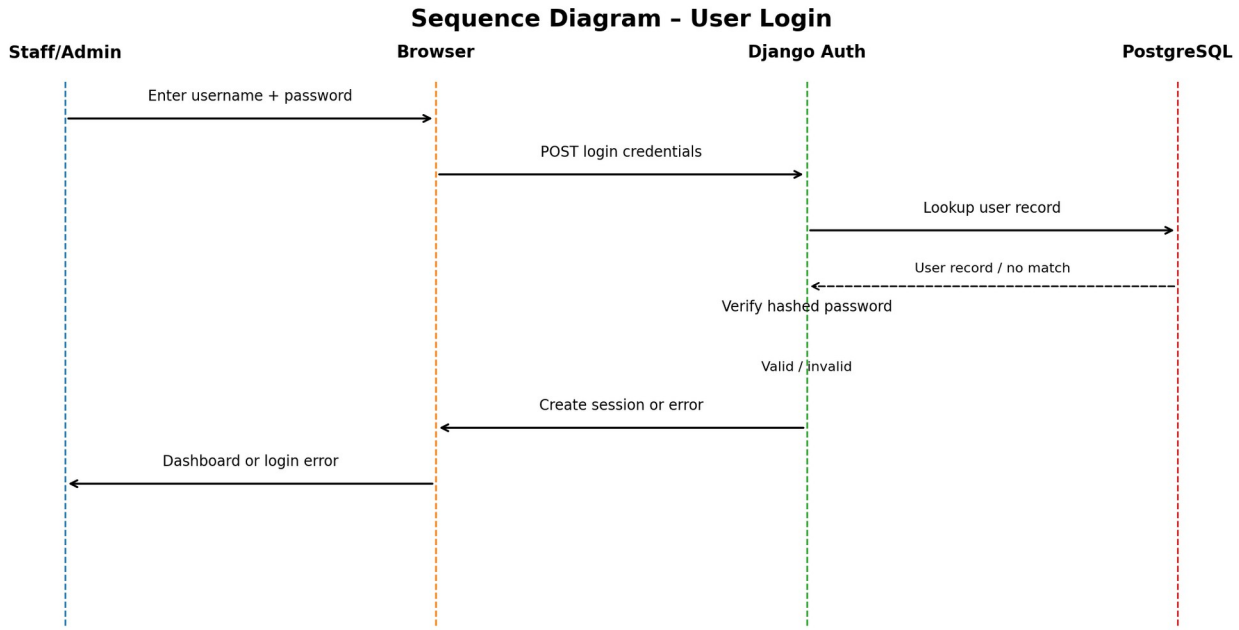


Figure 3. Login flow.

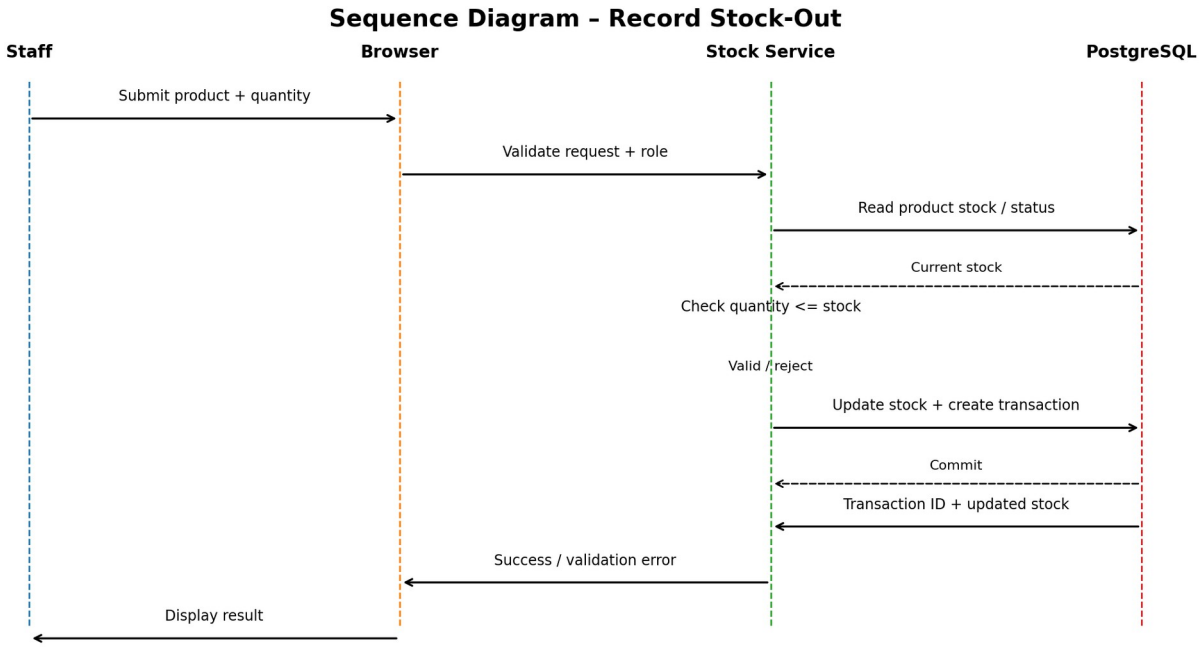


Figure 4. Stock-out flow.

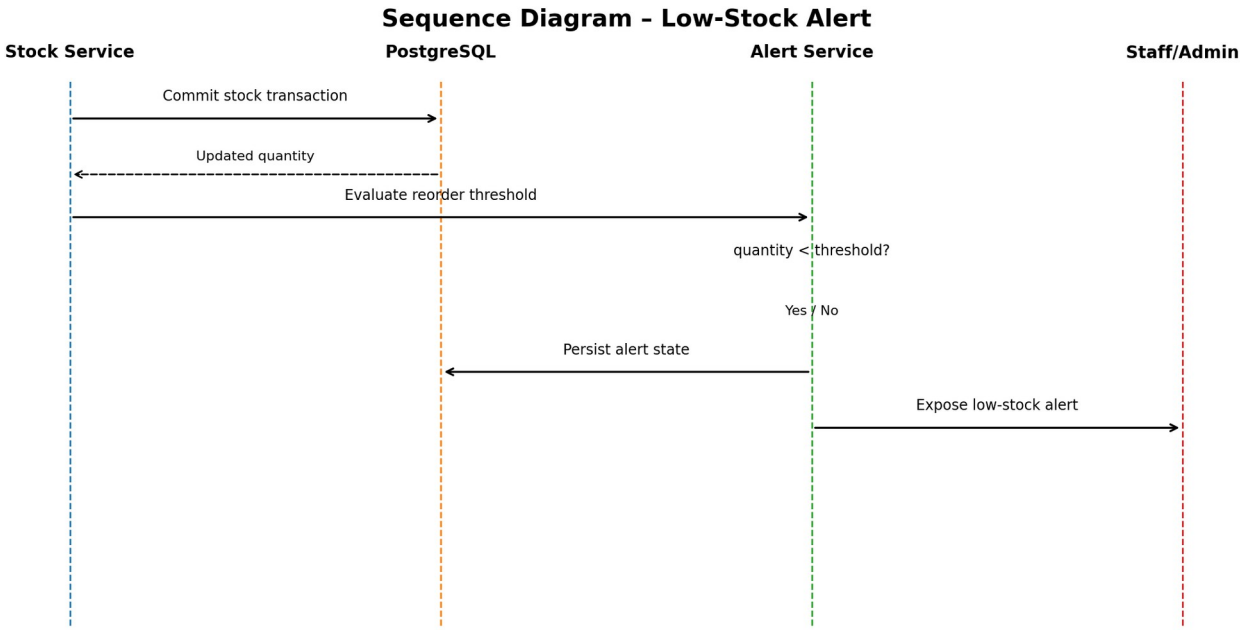


Figure 5. Low-stock alert flow.

4.3 API / Service Interface Design

The SRS permits an optional Django REST Framework API when a decoupled frontend is used. The following logical contracts define stable interfaces for the core operations; the same contracts can be implemented as Django service functions in a server-rendered deployment.

Method	Endpoint / Service	Purpose	Input	Success	Errors / Security
POST	/api/auth/login	Authenticate user	username/email, password	Session/token + role	401 invalid credentials; rate limiting; no

					password logging
GET	/api/products	Search/filter catalog	name/category/ SKU query	Product list + current stock	401 unauthenticated; validation of query parameters
POST	/api/stock-in	Record purchase	product_id, supplier_id, quantity	Transaction ID + new stock	400 validation; 403 role; 404 missing product/supplier; 409 conflict
POST	/api/stock-out	Record sale	product_id, quantity	Transaction ID + new stock	400 validation; 403 role; 409 insufficient stock
GET	/api/alerts/low-stock	View low-stock items	Authenticated user	Alert list	401 unauthenticated
GET	/api/reports/sales	Generate sales report	from_date, to_date	Aggregated report	403 admin-only; 400 invalid dates

API contract rules: validate mandatory fields; return stable identifiers; use appropriate HTTP status codes; enforce authorization server-side; never expose passwords or sensitive authentication data; document changes before dependent modules consume a new contract.

4.4 Error Handling, Logging & Monitoring

- Validation errors return clear field-level messages without exposing implementation details.
- Authentication failures use generic messages so that account existence is not unnecessarily disclosed.
- Authorization failures are distinguished from authentication failures internally and are logged as security events.
- Stock operations use database transactions and roll back on failure so quantity and transaction history cannot diverge.
- Duplicate submissions are detected through transaction/idempotency safeguards and database uniqueness.
- Logs record timestamp, event type, user identifier where appropriate, request outcome and correlation information; passwords and other sensitive credentials are never logged.
- Operational monitoring tracks application availability, search response time, transaction failures, failed logins and unauthorized access attempts.

4.5 UX Design

- Persistent navigation differs by role: Staff sees catalog, stock operations and alerts; Administrators additionally see product/category, supplier, staff-account and reporting functions.
- Dashboards surface current low-stock items and important operational messages.
- Forms use validation messages and confirmation dialogs for destructive actions such as disabling products.
- Stock-in and stock-out flows are designed to fit within the SRS limit of four screens/steps.
- Consistent Bootstrap layouts and terminology are used across catalog, stock and supplier screens.

4.6 Database / Data Integrity Design

The relational model centers on Product, Category, Supplier, User and StockTransaction. Products are disabled rather than hard-deleted, while stock transactions are retained indefinitely for audit. A stock transaction references an active product; stock-out quantity cannot exceed available quantity; transaction IDs are unique; supplier association is required for stock-in.

Entity	Key fields	Relationships	Integrity rule	Retention
User	user_id, username, role, password_hash	records StockTransaction	unique login; role constrained	Account lifecycle
Category	category_id, name, status	1-to-many Product	unique/valid category	Persistent
Product	product_id, SKU, price, quantity, threshold, status	belongs to Category; has Transactions/Alerts	quantity >= 0; disabled products cannot be sold	Soft delete
Supplier	supplier_id, name, contact, status	associated with StockTransaction	stock-in requires valid supplier	Persistent
StockTransaction	transaction_id, product, supplier, user, type, quantity, timestamp	references Product/User/Supplier	unique ID; quantity > 0; immutable history	Indefinite

4.7 Deployment Design

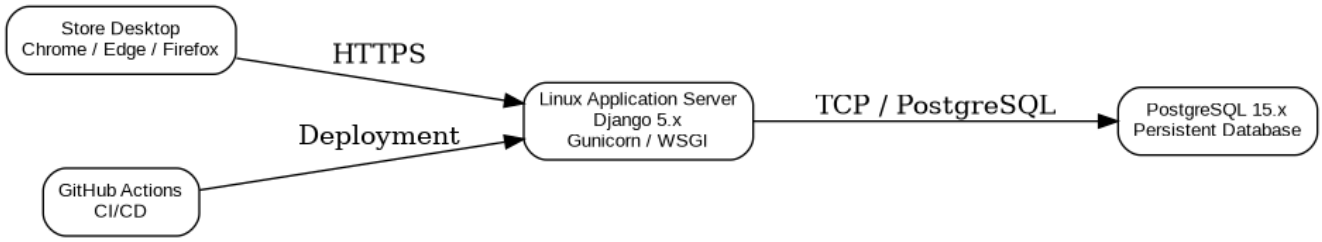


Figure 6. Logical deployment view for the Version 1.0 web application.

4.8 Open Issues & Next Steps

- Finalize exact API endpoints only if a decoupled frontend is selected; otherwise use the same service contracts internally.
- Add production deployment hardening, database backup/restore procedures and operational monitoring configuration.
- Perform security testing and performance testing against the implemented build before final sign-off.
- Future versions may consider multi-branch inventory, barcode/QR integration, POS integration or demand forecasting, all of which are out of scope for Version 1.0.

5. Appendices

5.1 Glossary

- RBAC – Role-Based Access Control.
- SKU – Stock Keeping Unit.
- Stock-in – purchase transaction increasing stock.
- Stock-out – sales transaction decreasing stock.
- RTM – Requirements Traceability Matrix.

5.2 References

- ESIMS SRS Team 8, Version 1.0.
- PES University Software Engineering Level 3 Student Submission Guide / Coffee Kiosk exemplar.
- PES University Software Engineering Mini-Project Deliverables Part-1.

5.3 Tools

- Draw.io / Graphviz / PlantUML for architecture and UML diagrams.
- Postman for API testing if REST endpoints are enabled.
- Django test framework / pytest and GitHub Actions for automated testing and CI.